VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CAT II, May 2023

B. Tech – Winter Semester 2022-2023

G1 slot answer key

Course Code : BCSE102L **Duration** : 90 Minutes

Course Title : Structured and Object Oriented Programming **Max. Marks** : 50

Slot : G1 **Class Numbers**: Common to G1

General Instructions: While answering programming questions, students are instructed to write the main method also.

Answer all the questions

Q.NO	QUESTION	Marks	Module	CO	BL

1	Create a structure called bank. Let the data members be Account_number, Name, Balance, Address. Consider Address data field to have houseno,city,pincode (structure within structure) details respectively.
	Write the C program to handle the following requirements:
	(a) Write a *function* search() that accepts "search_pincode" as argument and to print the details of the customer who belong to this "search_pincode" and have balance below Rs. 100.
	(b) Write a *function* transaction() to handle customer requests and update Balance accordingly. Assume that the request is received in the given format:
	Account_number,amount,transaction_code
	Note: Consider transaction_code to be (1 for deposit, 0 for withdrawal)
	Answer :
	
	#include <stdio.h>
	#include <stdlib.h>
	#include <string.h>
	
	struct address {
	int house_no;
	char city[20];
	int pincode;
	};
	
	struct bank {
	int account_number;
	char name[20];
	float balance;
		10	4	CO2	L3

```
    struct address addr;
} b[100];

void search(int search_pincode) {
    for (int i = 0; i < 100; i++) {
        if (b[i].addr.pincode == search_pincode &&
b[i].balance < 100) {
            printf("Account Number: %d\n",
b[i].account_number);
            printf("Name: %s\n", b[i].name);
            printf("Balance: %.2f\n", b[i].balance);
            printf("Address:\n");
            printf("\tHouse No.: %d\n", b[i].addr.house_no);
            printf("\tCity: %s\n", b[i].addr.city);
            printf("\tPincode: %d\n", b[i].addr.pincode);
        }
    }
}

void transaction(int account_number, float amount, int
transaction_code) {
    for (int i = 0; i < 100; i++) {
        if (b[i].account_number == account_number) {
            if (transaction_code == 1) {
                b[i].balance += amount;
                printf("Amount deposited successfully!\n");
                printf("New Balance: %.2f\n", b[i].balance);
            } else if (transaction_code == 0) {
                if (b[i].balance - amount >= 0) {
                    b[i].balance -= amount;
                    printf("Amount withdrawn successfully!\n");
                    printf("New Balance: %.2f\n", b[i].balance);
                } else {
                    printf("Insufficient balance!\n");
                }
            }
        }
    }
}

int main() {
    // Sample data
    struct address addr1 = {1234, "Seattle", 98101};
    struct address addr2 = {5678, "Redmond", 98052};
    struct address addr3 = {9101, "Bellevue", 98004};

    struct bank b1 = {1, "John Doe", 50.00, addr1};
    struct bank b2 = {2, "Jane Doe", 150.00, addr2};
    struct bank b3 = {3, "Bob Smith", 200.00, addr3};

    // Add sample data to array
    b[0] = b1;
    b[1] = b2;
    b[2] = b3;

    // Test search function
    search(98101);
```

	// Test transaction function
	transaction(1, 50.00, 1);
	
	return 0;
	}				

2	In C++, explain through your own program to demonstrate the following concepts,
	
	a) In what way we can automatically initialize the values to the *data member* of the class and how the parameter gets the input value at the time of object creation. (4 marks)
	Answer:
	In C++, you can use constructors to initialize the values of data members of a class automatically when an object is created. Constructors are special class members that are called by the compiler every time an object of that class is instantiated
	#include <iostream>
	using namespace std;
	
	class MyClass {
	public:
	int x;
	string y;
	
	// Constructor
	MyClass(int a, string b) {
	x = a;
	y = b;
	}
	};
	
	int main() {
	// Create an object of MyClass
	MyClass obj(10, "Hello");
	
	// Print values of data members
	cout << "x = " << obj.x << endl;
	cout << "y = " << obj.y << endl;
	
	return 0;
	}
	b) Consider the way where *one member function* contains multiple parameters. Out of which, atleast one argument value is default. (4 marks)
	Answer:
	In C++, you can define a member function with default arguments by specifying a default value for one or more of its parameters in the function declaration
	#include <iostream>
	using namespace std;
	
	class MyClass {
	public:
	void myFunction(int x, int y = 0, int z = 0) {
	cout << "x = " << x << endl;
		10	5	CO3	L2

	cout << "y = " << y << endl;
	cout << "z = " << z << endl;
	}
	};
	
	int main() {
	// Create an object of MyClass
	MyClass obj;
	
	// Call myFunction with one argument
	obj.myFunction(10);
	
	// Call myFunction with two arguments
	obj.myFunction(10, 20);
	
	// Call myFunction with three arguments
	obj.myFunction(10, 20, 30);
	
	return 0;
	}
	c) Explain the way by which the compiler can delete the memory created by the object implicitly. (2 marks)
	Answer : Destructor or delete keyword can be used to delete the memory allocated by the objects
	In C++, when you allocate memory for an object using the new operator, you must deallocate it using the delete operator. Using the delete operator on an object deallocates its memory. When delete is used to deallocate memory for a C++ class object, the object's destructor is called before the object's memory is deallocated (if the object has a destructor). If you don't call delete on an object created with new, it will remain in memory until the program ends				

3	i) *Complete* the following C++ program to access the private member of the given class "Point" (5 Marks)
	include<iostream>
	#include<math.h>
	using namespace std;
	class Point
	{private: int x; int y;
	
	Point(int a, int b)
	{ x = a; y = b; }
	
	……..//fill the remaining code…….
	
	#include<iostream>
	#include<math.h>
	using namespace std;
	class Point
	{
	private:
	int x;
	int y;
	friend void display(Point p); // Friend function declaration
	friend class AnotherClass; // Friend class declaration
		10	5	CO3	L3

```
        Point(int a,int b)
        {
            x = a;
            y =b;
        }
};

// Friend function definition
void display(Point p)
{
    cout<<"x = "<<p.x<<endl;
    cout<<"y = "<<p.y<<endl;
}

// Friend class definition
class AnotherClass
{
    public:
        void showPoint(Point p)
        {
            cout<<"x = "<<p.x<<endl;
            cout<<"y = "<<p.y<<endl;
        }
};

int main()
{
    Point p(10, 15);
    display(p);
    AnotherClass ac;
    ac.showPoint(p);
    return 0;
}
```

ii) Fix the error in the following code (5 Marks)

```
#include <iostream>
using namespace std;

class Example {
    public:
        static int count;
        int num;
        Example() {
            count++;
            num = 0;
        }
        static void printCount() {
            cout << "Count: " << count << endl;
        }
};

int count = 0;

int main() {
    Example ex1;
    Example ex2;
    Example ex3;

    ex1.num = 1;
```

	ex2.num = 2;
	ex3.num = 3;
	
	cout << "ex1.num: " << ex1.num << endl;
	cout << "ex2.num: " << ex2.num << endl;
	cout << "ex3.num: " << ex3.num << endl;
	
	Example.printCount();
	
	return 0;
	}
	Answer :
	Scope Resolution operator and class name to be used to intiliaze the variable and call the function				

4	Explain each of the following Inheritance model with suitable example
	a) Hybrid inheritance with ambiguity and Hybrid inheritance without ambiguity issue (6 marks)
	Answer:
	In C++, hybrid inheritance is a combination of more than one type of inheritance such as multilevel, multiple, or hierarchical. Ambiguity arises in hybrid inheritance when there are two or more functions with the same name and signature in different base classes. To resolve ambiguity in hybrid inheritance, we can use the scope resolution operator (::) to specify which function to use. Another way to resolve ambiguity is by using virtual base class,
	#include <iostream>
	using namespace std;
	
	class A {
	public:
	void display() {
	cout << "Class A" << endl;
	}
	};
	
	class B : public A {
	public:
	void display() {
	cout << "Class B" << endl;
	}
	};
	
	class C : public A {
	public:
	void display() {
	cout << "Class C" << endl;
	}
	};
	
	class D : public B, public C {
	public:
	void display() {
	cout << "Class D" << endl;
	}
		10	6	CO3	L2

	};
	
	int main() {
	D d;
	d.B::display();
	d.C::display();
	d.display();
	
	return 0;
	}
	b) Hierarchical inheritance (4 marks)
	Answer :
	In C++, hierarchical inheritance is when several classes are derived from a common base class. The feature of the base class is inherited onto more than one subclass.
	
	#include <iostream>
	using namespace std;
	
	class Animal {
	public:
	void info() {
	cout << "I am an animal." << endl;
	}
	};
	
	class Dog : public Animal {
	public:
	void bark() {
	cout << "I am a Dog. Woof woof." << endl;
	}
	};
	
	class Cat : public Animal {
	public:
	void meow() {
	cout << "I am a Cat. Meow meow." << endl;
	}
	};
	
	int main() {
	Dog d;
	d.info();
	d.bark();
	
	Cat c;
	c.info();
	c.meow();
	
	return 0;
	}				

5	Consider the example scenario of declaring the examination result. Design three classes: Student, Exam, and Result. The Student class has data members such as roll_number and name. Create the class Exam by inheriting from the student class. The Exam class has data members representing the *marks scored in six subjects*.	10	6	CO3	L3

Create the class Result by inheriting from the Exam class. The Result class has a member function computeResult() which will display “Pass” only if the student has scored above 50 in all the six subjects respectively. Implement the above scenario in C++ using a suitable inheritance model. Note: Use constructors appropriately in all the classes.

Answer :

```
#include <iostream>
using namespace std;

class Student {
public:
    int roll_number;
    string name;
};

class Exam : public Student {
public:
    int marks[6];
};

class Result : public Exam {
public:
    void computeResult() {
        int i;
        for (i = 0; i < 6; i++) {
            if (marks[i] < 50) {
                cout << "Fail" << endl;
                break;
            }
        }
        if (i == 6) {
            cout << "Pass" << endl;
        }
    }
};

int main() {
    Result r;
    r.roll_number = 1;
    r.name = "John";
    r.marks[0] = 60;
    r.marks[1] = 70;
    r.marks[2] = 80;
    r.marks[3] = 90;
    r.marks[4] = 85;
    r.marks[5] = 75;

    r.computeResult();

    return 0;
}
```